ShadowWatch Technical Documentation

ShadowWatch — Technical Documentation

A file-by-file reference for the ShadowWatch SOC Training Simulator codebase (PHP 8 / MySQL / vanilla JS).

---

1. Overview

ShadowWatch is a gamified Security Operations Center (SOC) training platform. Analysts log in, triage simulated alerts, manage incident tickets, analyze logs, look up threat intelligence, and earn points/rank on a leaderboard. The app follows a classic procedural-PHP structure: each top-level `.php` file is a page that pulls in shared includes (`session.php` , `auth.php` , `functions.php` , `header.php` / `footer.php`), queries the database directly via a PDO wrapper, and renders server-side HTML with small AJAX calls (`apiPost`) for interactive actions.

---

2. Architecture

Layer	Implementation
Language	PHP 8.x, procedural style, no framework
Database	MySQL 8.x via a PDO singleton (`Db.php`)
Frontend	Server-rendered HTML + vanilla JS (`assets/js/app.js` , not included in this file set)
Auth	Session-based, `password_hash()` / `password_verify()` , CSRF tokens on all POST forms
Config	Constants defined in `includes/config.php`

Request flow for a typical page (e.g. `dashboard.php`): 1. Sets `$pageTitle` / `$currentPage` , then requires `includes/header.php` , which in turn loads session/auth/config/db and renders the shared sidebar/topbar. 2. Runs one or more queries via the global `db()` helper. 3. Renders HTML inline, mixing PHP with markup. 4. Emits a `<script>` block calling small `apiPost()` -based handlers against an `/api/*.php` endpoint. 5. Requires `includes/footer.php` to close out the layout.

---

3. Page & Script Reference

3.1 Public / Unauthenticated Pages

`index.php`

Landing page. Redirects to `dashboard.php` if already logged in (`isLoggedIn()`), otherwise renders a marketing-style hero section with feature cards and links to `login.php` / `register.php` / `about.php` .

`about.php`

Public marketing/about page. Pulls live counts (active users, resolved alerts, active scenarios, incidents) directly from the DB for display as "stats," lists the eight platform modules (Alert Console, Attack Scenarios, Incident Management, Log Analyzer, Threat Intelligence, Gamification, Admin Panel, Secure Authentication), and shows a 4-step "How it works" section. Nav adapts based on `isLoggedIn()` .

`login.php`

Handles `GET` (render form) and `POST` (authenticate). On POST: validates the CSRF token via `validateCsrfToken()` , then calls `loginUser($username, $password)` from `includes/auth.php` . On success, redirects to `?redirect=` or `/shadowwatch/dashboard.php` ; on failure, re-renders the form with `$error` . Displays demo credentials (admin/jsmith/alee, all password `password`) directly on the page — convenient for training use but should be removed before any public/production deployment.

`register.php`

Handles account creation. Validates required fields, email format, username length/charset (`^[a-zA-Z0-9_]+$` , 3–30 chars), password confirmation, and minimum password length (8), then calls `registerUser()` . New accounts start as Junior Analyst per the on-page note.

3.2 Authenticated Application Pages

`dashboard.php`

Main SOC overview. Queries: - Aggregate stats (open/critical alerts, open incidents, alerts today, resolved today, active users) via a single multi-subquery `SELECT` . - 8 most recent alerts (joined to `scenarios` for scenario name). - 5 most recent incidents (joined to `users` for creator name). - Top 5 leaderboard entries via `getLeaderboard(5)` . - The current user's own stats and `getLevelProgress()` . - Open-alert severity distribution, rendered as colored progress bars.

Renders: stat cards, a recent-alerts table, a severity-distribution widget, a "My Progress" XP panel, a "Top Analysts" mini-leaderboard, and a recent-incidents table. Row clicks navigate to the relevant detail page via inline `onclick` handlers.

`alerts.php` — Alert Console

- Reads `severity` , `status` , `category` filters and an `id` from `$_GET` , sanitizes them, and builds a dynamic `WHERE` clause (parameterized) to list up to 100 alerts, most severe/newest first.
- If `id` is set, loads that alert's full detail (joined to its scenario).
- Renders a filterable/searchable table plus a detail side-panel with an **Acknowledge** action, a **Resolve** modal (choose a response action — Block IP, Isolate Host, Reset Password, Patch System, Update Firewall Rules, Run Full Scan, Monitor & Watch, Mark False Positive — each worth different points), and a **Create Incident** modal.
- Client-side JS (`ackAlert` , `submitResolve` , `submitIncident` , `generateAlert`) posts to `/api/resolve_alert.php` , `/api/create_incident.php` , and `/api/generate_alert.php` respectively (API layer not included in this file set).
- Polls `updateAlertCount()` every 30s (function presumably defined in `app.js`).

`logs.php` — Log Analyzer

- Filters `system_logs` by `type` (firewall/auth/web/dns/endpoint/network/system) and supports selecting a single log by `id` .
- Shows aggregate stats: total logs, malicious count, analyzed count, and accuracy (`correct / analyzed`).
- Detail panel lets an analyst submit a verdict (`submitAnalysis()` → `/api/fetch_alerts.php` with `action: analyze_log`), and extracts IOCs from the raw log text client-server-side using regex:
  - IPs: `/\b(?:\d{1,3}\.){3}\d{1,3}\b/`
  - Domains: `/\b[a-z0-9-]+\.[a-z]{2,}\b/i`
  - Hashes: `/\b[0-9a-f]{32,64}\b/`
- Each extracted IOC is clickable and triggers `checkThreat()` , which posts to `/api/fetch_alerts.php` (`action: threat_lookup`) and renders a found/not-found result in a modal.
- Includes a "Generate Training Logs" modal (type, malicious mix, count) that posts `action: generate_logs` .

`incidents.php` — Incident Management

- Lists all incidents ordered by status priority (`open → investigating → pending → resolved → closed`) then severity then

recency, each joined to creator and assignee usernames.
- Supports a detail view (`?id=`) and a create modal (`?create=1` auto-opens it).
- Status can be updated inline (`updateIncidentStatus()`), and — gated behind `isSeniorAnalyst()` — incidents can be reassigned (`assignIncident()`). Both call `/api/create_incident.php` with different `action` values (`update_status` , `assign`).
- Creating an incident (`createIncident()`) posts `action: create` with title/severity/category/description/assignee.

`scenarios.php` — Threat Intelligence

Three tabs, switched via `?tab=` : 1. **IOC Database** (`intel` , default) — lists active `threat_intel` rows sorted by confidence/recency, with searchable table and three summary breakdowns (by indicator type, by confidence, by top threat type). 2. **IOC Lookup** (`lookup`) — a form that POSTs an indicator, looks it up with an exact-match query (`WHERE indicator = ?`), and displays a "found" (with description/tags) or "not found" result. Also renders a scrolling "quick test" list of known IOCs and a terminal-style feed of all IOCs. 3. **Attack Scenarios** (`scenarios`) — a card grid of scenarios grouped by difficulty (beginner/intermediate/advanced/expert), each showing severity, category icon, description, and point value.

> **Schema note:** this page's IOC query uses columns `indicator` , `indicator_type` , `confidence` , `tags` , `last_seen` , `first_seen` , `threat_type` , `description` on a table it refers to as `threat_intel` . The provided `Schema.sql` defines that table with different column names (`ioc_type` , `value` instead of `indicator_type` / `indicator`). If you're wiring this page up against the provided schema, reconcile these names first — they're documented here as-written in the code, not the schema.

`leaderboard.php`

Ranks all active users by `score` descending, with a podium (top 3, medal styling) and a full sortable table (rank, analyst, level, score, alerts resolved, incidents created, accuracy — accuracy is currently hard-coded to `0` in both the table and the "My Stats" panel; wire this up to real data before relying on it). Also shows the current user's rank, level progress bar, a 2×3 stat grid, and the 15 most recent `score_events` platform-wide.

`profile.php`

Shows a given user's profile (`?id=` , defaults to the logged-in user via `requireLogin()`). Displays avatar/role badge, level + XP progress bar, join date/last-active, four stat cards (alerts resolved, incidents created, logs analyzed, badges earned), a badge grid (earned vs. locked, based on `user_badges` / `badges` tables), and a 20-item recent activity feed from `score_events` .

`settings.php`

Four independent forms, all POSTing to the same page with a `form_action` discriminator, each validated with `validateCsrfToken()` : - `update_profile` — change email (checked for uniqueness and valid format). - `change_password` — requires current password verification (`password_verify`), 8-char minimum, confirmation match; re-hashes with `password_hash(..., PASSWORD_DEFAULT)` . - `update_preferences` — three boolean toggles (alert sound, compact view, auto refresh) stored as a JSON blob in `users.preferences` . - Also displays a read-only "Account Info" panel (ID, score, last login, active status). - All successful actions call `logActivity()` for an audit trail.

3.3 Core / Include Files

`Config.PHP`

Central constants: DB connection (`DB_HOST/PORT/NAME/USER/PASS/CHARSET`), app metadata (`APP_NAME` , `APP_VERSION` , `APP_URL` , `APP_ROOT`), session settings (`SESSION_NAME` , 8-hour `SESSION_LIFETIME` , `SESSION_SECURE`), brute-force protection (`MAX_LOGIN_ATTEMPTS = 5` , `LOCKOUT_DURATION = 900s`), upload/log directories, `HASH_COST = 12` for bcrypt, and an `ENV` flag that toggles error display (`display_errors` on for `development` , fully suppressed for anything else). Sets UTC as the default timezone.

> Note: `DB_PASS` is empty and `ENV` defaults to `'development'` (verbose error output) — both fine for a local training instance, both things to change before any non-local deployment.

`Db.php`

A minimal PDO singleton:

```
        class Database {
```

```
        private static ?PDO $instance = null;
        public static function getInstance(): PDO { ... }
    }
    function db(): PDO { return Database::getInstance(); }
```

Connects lazily on first use with `ERRMODE_EXCEPTION`, `FETCH_ASSOC` as the default fetch mode, and `EMULATE_PREPARES` disabled (so parameter binding is handled natively by the MySQL driver, not emulated by PHP — this is what makes the parameterized queries throughout the app resistant to SQL injection). On connection failure, logs the error server-side and returns a generic JSON error with a 500 status rather than leaking DB credentials/errors to the client.

> The rest of the codebase calls methods like `db()->fetchAll(...)`, `db()->fetchOne(...)`, and `db()->execute(...)` — convenience wrappers around `PDO::prepare` / `execute` / `fetch(All)` that aren't included in this file set (likely defined via a subclass or trait alongside `Database`, not shown here).

3.4 Development / Maintenance Utilities

These four files are developer tools, not application pages. **They should not ship to a production or public-facing environment.**

`debug_login.php`

Fetches the `admin` user's stored hash and runs `password_verify('password', $hash)`, printing the result. Useful for confirming demo credentials are correctly hashed in the DB — but it echoes the actual `password_hash` value to anyone who loads the page, which is a credential-exposure risk outside a local dev environment.

`debug_dash.php`

Forces `$_SESSION['user_id'] = 1` (i.e., logs the requester in as user #1 with no authentication) and then walks through `getCurrentUser()`, `getLevelProgress()`, and `getOpenAlertsCount()`, echoing progress at each step. This is an **authentication bypass** if left reachable outside local development — anyone hitting this URL is silently logged in as user 1 (typically the admin seed account).

`fix_passwords.php`

Resets **every** user's password to the literal string `password` in a single unparameterized-scope `UPDATE users SET password_hash = ?` (no `WHERE` clause), then prints the new hash to the page. The file's own comment says "Delete this file after running!" — that instruction should be followed immediately; left in place, it's a standing "reset everyone's password" endpoint reachable by anyone who knows the URL.

`Schema.sql`

Full DDL for the `shadowwatch` database plus seed data. Tables: `users`, `scenarios`, `alerts`, `incidents`, `incident_comments`, `logs`, `threat_intel`, `achievements`, `user_achievements`, `activity_log`, `settings`. See §4 below.

---

4. Database Schema Summary (`Schema.sql`)

Table	Purpose	Key columns
`users`	Accounts	`role` (analyst/senior_analyst/team_lead/admin), `rank_title`, `xp`, `level`, `is_active`
`scenarios`	Attack-scenario templates	`difficulty` (easy/medium/hard/critical), `category`, `xp_reward`, `time_limit_minutes`
`alerts`	Simulated security alerts	`severity`, `status` (open/investigating/resolved/false_positive), `source_ip` / `dest_ip`, FK → `scenarios`, `assigned_to`, `resolved_by`
`incidents`	Incident tickets	`ticket_number` (unique), `status` (open/in_progress/pending/resolved/closed), FK → `alerts`, `assigned_to`, `created_by`

`incident_comments`	Ticket comment thread	FK → `incidents`, `users`
`logs`	Simulated system logs	`log_type` (syslog/auth/firewall/ids/web/dns/dhcp/endpoint), `is_malicious`
`threat_intel`	IOC database	`ioc_type` (ip/domain/hash/url/email), `value`, `confidence` (low/medium/high)
`achievements` / `user_achievements`	Badge system	`condition_type`, `condition_value`, join table with a unique `(user_id, achievement_id)` constraint
`activity_log`	Audit trail	`action`, `entity_type` / `entity_id`, `ip_address`
`settings`	App-wide key/value config	`xp_per_alert`, `xp_per_incident`, lockout settings, etc.

Seed data includes 3 users (`admin`, `analyst01`, `soc_lead` — all sharing one bcrypt hash, i.e. one shared password), 6 scenarios, 8 achievements, and 10 threat-intel entries (Tor exit node, C2 domain, phishing domain, an EICAR test hash, etc.).

> As noted in §3.2, some application pages (`scenarios.php`, `leaderboard.php`, `logs.php`) query column names (`indicator`, `score`, `analysed_by`, `verdict`, `system_logs`) that don't exactly match this schema (`value`, `xp`, no `analysed_by` / `verdict` columns, table named `logs` not `system_logs`). This documentation reflects what's in each file as provided; if you're assembling the full app, you'll want a single schema version that matches the PHP queries exactly.

---

5. Security Notes

What's implemented well: - All DB access goes through parameterized PDO queries with emulated prepares disabled — no string-concatenated SQL was found in any reviewed file. - Passwords are hashed with `password_hash()` /verified with `password_verify()`, never stored or compared in plaintext. - CSRF tokens are generated and validated on every state-changing form (`login`, `register`, `settings`). - Output is consistently escaped with `htmlspecialchars()` (via the app's `sanitize()` helper) before being echoed into HTML. - Role checks (`isSeniorAnalyst()`, etc.) gate sensitive actions like incident reassignment.

What to fix before any deployment beyond a local/personal training instance: 1. Delete or environment-gate `fix_passwords.php`, `debug_login.php`, and `debug_dash.php` — each is a live credential/auth-bypass risk if reachable. 2. Set `ENV` to `'production'` in `Config.PHP` outside local development so PHP errors aren't rendered to visitors. 3. Set a real `DB_PASS` and set `SESSION_SECURE` to `true` once served over HTTPS. 4. Reconcile the schema/query naming mismatches noted in §3.2/ §4 so lookups don't silently fail.

---

6. Setup

1. `git clone` the repo into your web root (WAMP/XAMPP/MAMP path).
2. Create the `shadowwatch` database and import `Schema.sql`.
3. Edit `includes/config.php` (`Config.PHP`) with your local DB credentials.
4. Visit the app root; register a new analyst or use a seeded demo account.
5. **Delete `fix_passwords.php`, `debug_login.php`, and `debug_dash.php`** once initial setup/testing is done.

Generated from the provided source files: *`alerts.php`, `logs.php`, `about.php`, `Config.PHP`, `dashboard.php`, `Db.php`, `debug_dash.php`, `debug_login.php`, `fix_passwords.php`, `incidents.php`, `index.php`, `leaderboard.php`, `login.php`, `profile.php`, `register.php`, `scenarios.php`, `Schema.sql`, `settings.php`.*